

CSE461: Mobile Computing

Project Guidelines

Spring 2026

Overview

Each team must propose a unique Android application project built using Kotlin. The project should address a well-defined, real-world problem and demonstrate the team's ability to design and implement a complete, production-quality Android app. All projects must follow the MVVM (Model–View–ViewModel) architectural pattern, use Fragments for screen navigation, persist data locally with a Room Database, and design all layouts using XML.

1. Project Scope

Your Android application must address a meaningful real-world problem and must be built entirely using the following mandatory technology stack:

- Language: Kotlin (no Java permitted)
- Architecture: MVVM — Model, View (Activity/Fragment), ViewModel, Repository
- Local Database: Room Database with at least two related entities and DAO interfaces
- UI Navigation: Fragments managed via the Android Navigation Component or FragmentManager
- Layouts: All UI screens designed in XML (no Jetpack Compose)

Projects that extend the stack with additional libraries (e.g., Retrofit for networking, LiveData/Flow for reactive streams, Hilt for dependency injection, RecyclerView with DiffUtil) are strongly encouraged and will receive additional credit in the Architecture & Implementation criterion.

2. Project Requirements

2.1 Real-World Problem Focus

- Identify and clearly define a significant real-world problem addressable by an Android app.
- Explain why a mobile solution is appropriate and who the target users are.
- Describe how the app integrates into the daily context of those users.
- Discuss any accessibility, usability, or societal considerations relevant to the problem.

2.2 MVVM Architecture & Design Justification

- Provide a clear architectural diagram showing all MVVM layers: View (Fragments/Activities), ViewModel, Repository, Room Database, and any remote data sources.
- Explain the responsibility of each layer and how they communicate (LiveData, StateFlow, Coroutines).
- Justify your Fragment structure: describe every Fragment, its role, and how navigation between Fragments is handled.
- Describe your Room Database schema: entity classes, relationships (one-to-many, many-to-many), and DAO query methods.
- Discuss how the ViewModel survives configuration changes and why no direct database access occurs in the View layer.
- Address alternative design decisions considered and why MVVM was the correct choice for your use case.

2.3 Implementation

- The project must be a native Android application written entirely in Kotlin.
- All UI layouts must be implemented in XML using ConstraintLayout, LinearLayout, or RelativeLayout as appropriate — no programmatic layout inflation or Jetpack Compose.
- Room Database must include: at least two @Entity classes, at least one @Dao interface with CRUD operations, and one @Database class.
- MVVM must be properly layered: Fragments observe LiveData/StateFlow from ViewModel; ViewModel calls Repository; Repository interacts with Room DAO.
- Use Android Jetpack libraries: ViewModel, LiveData (or StateFlow/SharedFlow), Navigation Component, and Room.
- You may additionally use: Retrofit, Glide/Picasso, Hilt/Koin, RecyclerView, ViewBinding, DataBinding, or WorkManager.
- Code must be original, clearly commented, and fully buildable in Android Studio by the instructor.
- Include a README.md with setup instructions, minimum SDK version, and any required API keys (use placeholders — never commit real secrets).

2.4 Performance & Quality Analysis

For all projects:

- Test on at least two screen configurations (e.g., phone and tablet, or two different API levels) using the Android Emulator.
- Profile memory usage with Android Studio Profiler; identify and address any memory leaks (use LeakCanary if applicable).
- Ensure smooth RecyclerView scrolling (target ≥ 60 fps) if lists are used; demonstrate use of DiffUtil.

- Handle configuration changes (rotation) correctly — no data loss on screen rotation due to proper ViewModel usage.
- Implement proper error handling for database operations using Kotlin Coroutines (try/catch, Result, or sealed classes).

For projects with remote data (optional extension):

- Handle network failures gracefully with appropriate user feedback (loading states, error messages).
- Implement an offline-first strategy: cache remote data in Room and serve from local DB when offline.

2.5 Code Quality & Android Best Practices (Required — 2026)

All projects must demonstrate the following Android development standards:

- ViewBinding or DataBinding: No findViewById() calls — all view references through ViewBinding.
- Coroutines: All Room database and network calls executed on background threads using Kotlin Coroutines and appropriate Dispatchers (IO, Default).
- Lifecycle Awareness: Observers and coroutine scopes properly tied to Fragment/Activity lifecycle to prevent leaks.
- Resource Management: All strings in strings.xml, dimensions in dimens.xml, colors in colors.xml — no hardcoded values in XML layouts.
- Manifest Permissions: Declare and request only necessary permissions; handle runtime permission results properly.

2.6 Presentation

- Prepare a clear and visually compelling presentation summarizing your work.
- Cover: problem motivation, app overview, MVVM architecture walkthrough, Room schema, Fragment navigation flow, live demo, and key insights.
- Include a live demo of the running application on an emulator or physical Android device.
- Walk through at least one complete user flow end-to-end during the demo.
- Be prepared for in-depth Q&A on your architecture and implementation decisions.
- Presentations will be 12–15 minutes per team followed by a 5-minute Q&A.

3. Submission Deliverables

Deliverable	Description	Format
Full Source Code	Complete Android Studio project, well-commented and buildable from source	GitHub repo (university email)

Deliverable	Description	Format
Presentation Slides	Covers problem, MVVM architecture, Room schema, Fragment flow, demo highlights, and results	PDF or PPTX
Written Report	Problem statement, architecture diagram, Room schema, Fragment map, performance findings, code quality section, and challenges	PDF (4–8 pages)
APK Demo Build	Signed debug APK of the final application runnable on Android 8.0+ (API 26+)	.apk file
Video Walkthrough	Screen recording demonstrating all major features and at least one complete user flow	MP4 or YouTube link
Contribution Log	Each member specifies their role and individual tasks completed	Included in report
GitHub Repository Link	Public repository using your university email account (not a personal GitHub account)	GitHub URL

4. Team Composition

- Teams must consist of 3–5 members, with 4 members being the preferred size.
- Each team member must clearly specify their role and individual contribution in the Contribution Log.
- Suggested roles: Project Lead, UI/UX & XML Layouts, ViewModel & Business Logic, Room Database & Repository.
- All members are expected to understand the entire project and be able to answer questions about any part.

5. Originality & Academic Integrity

Projects that replicate work from tutorials, open-source sample apps, or published sources without significant original contribution will be disqualified, resulting in a grade of zero and a report of academic dishonesty.

Use of AI tools (e.g., GitHub Copilot, ChatGPT) for code assistance is permitted provided:

- All AI-assisted code is clearly annotated with a comment indicating AI assistance (e.g., // AI-assisted).
- The team fully understands and can explain every line of code submitted.
- The core architecture design, Room schema design, Fragment navigation logic, and written report are the team's own original work.

6. Key Questions to Prepare For

During the Q&A, you must be able to confidently answer questions such as:

- Walk us through your MVVM layers — what is the responsibility of each layer in your app?
- How does your Fragment communicate with the ViewModel, and why is this approach correct?
- Describe your Room Database schema: what entities exist, what are their relationships, and how are they queried?
- What happens to your ViewModel and its data when the screen is rotated?
- How do you handle background threading for database operations, and which Dispatcher do you use and why?
- Why did you structure your Fragment navigation the way you did? What alternatives did you consider?
- What are the failure modes of your app, and how does it behave when Room operations fail?

7. Grading Criteria

Criterion	Weight	Description
Originality & Problem Relevance	10%	Novel, meaningful real-world problem that genuinely benefits from an Android mobile solution
MVVM Architecture & Design	20%	Correct and clean MVVM layering, justified Fragment structure, clear architecture diagram, proper Room schema design
Implementation Quality	20%	Kotlin code quality, correct Room usage, ViewBinding, Coroutines on IO thread, no hardcoded resources, buildable project
Technical Understanding	20%	Depth of explanation during Q&A; ability to discuss MVVM internals, Room queries, Fragment lifecycle, and design trade-offs
Deliverable Completeness	15%	All required artifacts submitted on time, clearly organized, and meeting stated technical requirements
Discussion & Presentation	15%	Clear communication, live demo quality, visual quality of slides, and Q&A performance

Good luck — build something great with Kotlin!